

## Contents

|                                                           |    |
|-----------------------------------------------------------|----|
| Proposed UI Changes (XAML) .....                          | 1  |
| Change Inventory .....                                    | 1  |
| Module_Receiving/Views/View_Receiving_EditMode.xaml ..... | 2  |
| Module_Reprint/Views/View_Reprint_ModulePage.xaml .....   | 2  |
| Shared .....                                              | 2  |
| Part 1 – EditMode Toolbar and Selection Bar .....         | 3  |
| Toolbar: the Batch Edit button .....                      | 3  |
| Selection Bar .....                                       | 3  |
| Part 2 – EditModeDataGrid Multi-Select .....              | 5  |
| Selection mode .....                                      | 5  |
| The synchronization contract in code-behind .....         | 5  |
| Tri-state select-all header .....                         | 5  |
| Pending-change marker .....                               | 6  |
| Part 3 – Batch Edit Overlay Wireframe .....               | 7  |
| Width adaptivity .....                                    | 9  |
| Dialog result handling .....                              | 9  |
| Part 4 – Reprint Batch Filter Chips .....                 | 9  |
| Two new columns .....                                     | 10 |
| New Converters and Resources .....                        | 10 |
| XAML Conventions and Risks .....                          | 11 |
| Risks .....                                               | 11 |

## Proposed UI Changes (XAML)

Last Updated: 2026-10-02

Companion to docs/specs/BatchSkidEdit\_UX\_Specification.md and docs/specs/BatchReprint\_Automation\_Spec.md. This file names the exact controls to add or change, so the implementation pass does not have to rediscover the seams.

All anchors are given by x:Name or by structural position rather than by line number, because View\_Receiving\_EditMode.xaml is actively edited and line numbers do not survive a week.

## Change Inventory

Two authoring rules keep these tables legible in a PDF, and both matter more than they look. A cell cannot be wider than its column's share of the page, and a long code span cannot be broken across lines, so a 50-character path in a narrow column bleeds over its neighbour. File paths are therefore lifted into the section headings, and the delimiter row of each table is weighted to match the width its column actually needs.

New files, all relative to Module\_Receiving/:

| # | File                               | Purpose                                         |
|---|------------------------------------|-------------------------------------------------|
| 1 | Dialog_Receiving_BatchEdit.xaml    | Batch edit overlay content                      |
| 2 | Dialog_Receiving_BatchEdit.xaml.cs | XamlRoot assignment and result plumbing only    |
| 3 | ViewModel_Receiving_BatchEdit.cs   | Overlay state, field model, reconciliation math |

Files 1 and 2 belong in Dialogs/; file 3 belongs in ViewModels/.

#### **Module\_Receiving/Views/View\_Receiving\_EditMode.xaml**

| # | Element or region                | Change                                                                        | Type   |
|---|----------------------------------|-------------------------------------------------------------------------------|--------|
| 4 | EditModeDataGrid                 | SelectionMode Single becomes Extended; add a SelectionChanged handler         | Modify |
| 5 | Checkbox column, first column    | Add a tri-state select-all header via HeaderTemplate; add an automation name  | Modify |
| 6 | New row between toolbar and grid | Insert the Selection Bar; add a RowDefinition and shift later Grid.Row values | Add    |
| 7 | Toolbar Edit group               | Add a Batch Edit button after ColumnsButton                                   | Add    |
| 8 | PartID column                    | Add a “pending batch change” marker in the cell template                      | Modify |
| 9 | Row container background         | Selected-state tint via DataGrid.RowStyle or the wired LoadingRow handler     | Modify |

#### **Module\_Reprint/Views/View\_Reprint\_ModulePage.xaml**

| #  | Element or region            | Change                                                        | Type   |
|----|------------------------------|---------------------------------------------------------------|--------|
| 10 | Filters row                  | Add a second chip row beneath the existing filters grid       | Add    |
| 11 | Grid above the history list  | Add a batch context InfoBar above the existing queued InfoBar | Add    |
| 12 | Header grid and row template | Widen to include Location and Heat/Lot                        | Modify |
| 13 | Footer                       | ReprintButtonLabel is already bound; no structural change     | Reuse  |

#### **Shared**

| #  | File                    | Change                                              | Type |
|----|-------------------------|-----------------------------------------------------|------|
| 14 | Module_Core/Converters/ | Add the converters listed in the converters section | Add  |

## Part 1 — EditMode Toolbar and Selection Bar

### Toolbar: the Batch Edit button

Inside the existing Edit group StackPanel (Grid.Column="2"), immediately after ColumnsButton. Reuse the exact 34×34 Button + FontIcon + caption pattern the group already uses, so the ribbon does not gain a visually foreign control.

```
<Button
    Width="34"
    Height="34"
    Padding="0"
    AutomationProperties.Name="Batch edit selected skids"
    Command="{x:Bind ViewModel.OpenBatchEditCommand}"
    ToolTipService.ToolTip="Batch edit the selected skids">
    <FontIcon Glyph="⦿" />
</Button>
```

Glyph E8C8 is a bulk-edit style glyph; confirm against the Segoe Fluent icon list during implementation. OpenBatchEditCommand carries CanExecute = SelectedCount > 0.

### Selection Bar

Insert as a new Grid.Row="1" element between the toolbar Border and EditModeDataGrid, and redeclare the row definitions as:

```
<Grid.RowDefinitions>
    <RowDefinition Height="Auto" /> <!-- 0: toolbar -->
    <RowDefinition Height="Auto" /> <!-- 1: selection bar -->
    <RowDefinition Height="*" /> <!-- 2: data grid -->
    <RowDefinition Height="Auto" /> <!-- 3: footer -->
</Grid.RowDefinitions>
```

This shifts EditModeDataGrid and the empty-state overlay from Grid.Row="1" to Grid.Row="2" and the footer to Grid.Row="3". That is the only ripple from this insert.

```
<Border
    Grid.Row="1"
    Margin="0,0,0,8"
    Padding="12,8"
    Background="{ThemeResource AccentFillColorDefaultBrush}"
    CornerRadius="6"
    Visibility="{x:Bind ViewModel.HasSelection, Mode=OneWay}">
    <Grid ColumnSpacing="12">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto" /> <!-- count -->
            <ColumnDefinition Width="*" /> <!-- totals -->
            <ColumnDefinition Width="Auto" /> <!-- expected total -->
            <ColumnDefinition Width="Auto" /> <!-- variance -->
            <ColumnDefinition Width="Auto" /> <!-- actions -->
        </Grid.ColumnDefinitions>

        <TextBlock
            Grid.Column="0"
            VerticalAlignment="Center"
            FontWeight="SemiBold"
            Text="{x:Bind ViewModel.SelectionHeadline, Mode=OneWay}" />
    </Grid>
</Border>
```

```

<TextBlock
    Grid.Column="1"
    VerticalAlignment="Center"
    Foreground="{ThemeResource TextFillColorSecondaryBrush}"
    Text="{x:Bind ViewModel.SelectionSummary, Mode=OneWay}"
    TextTrimming="CharacterEllipsis" />

<NumberBox
    Grid.Column="2"
    Width="150"
    Header="Expected total"
    SpinButtonPlacementMode="Hidden"
    PlaceholderText="Optional"
    Value="{x:Bind ViewModel.ExpectedTotal, Mode=TwoWay}" />

<StackPanel
    Grid.Column="3"
    Orientation="Horizontal"
    Spacing="6"
    VerticalAlignment="Bottom">
    <FontIcon
        FontSize="14"
        Foreground="{x:Bind ViewModel.VarianceBrush, Mode=OneWay}"
        Glyph="{x:Bind ViewModel.VarianceGlyph, Mode=OneWay}" />
    <TextBlock
        VerticalAlignment="Center"
        Foreground="{x:Bind ViewModel.VarianceBrush, Mode=OneWay}"
        Text="{x:Bind ViewModel.VarianceText, Mode=OneWay}" />
</StackPanel>

<StackPanel
    Grid.Column="4"
    Orientation="Horizontal"
    Spacing="6"
    VerticalAlignment="Center">
    <Button
        Command="{x:Bind ViewModel.OpenBatchEditCommand}"
        Content="Batch Edit..."
        Style="{StaticResource AccentButtonStyle}" />
    <Button
        Command="{x:Bind ViewModel.PrintBatchReportCommand}"
        Content="Print Report" />
    <Button
        Command="{x:Bind ViewModel.ReprintBatchCommand}"
        Content="{x:Bind ViewModel.ReprintBatchLabel, Mode=OneWay}" />
    <Button
        Command="{x:Bind ViewModel.ClearSelectionCommand}"
        Content="Clear" />
</StackPanel>
</Grid>
</Border>

```

Notes:

- Visibility binds directly to a bool, which works because WinUI 3 has a built-in BooleanToVisibility conversion for x:Bind. If the team prefers an explicit converter, use the already-registered BooleanToVisibilityConverter in this view's resources.
- Foreground on the bar uses accent fill; verify text contrast against TextOnAccentFillColorPrimaryBrush rather than leaving the default foreground.
- Recent Modified · 24h and the recency chips belong to the Reprint page, not here.

## Part 2 — EditModeDataGrid Multi-Select

### Selection mode

```
<controls:DataGrid
    x:Name="EditModeDataGrid"
    SelectionMode="Extended"
    SelectionChanged="EditModeDataGrid_SelectionChanged"
... />
```

SelectedItem stays bound to ViewModel.SelectedLoad (Mode=TwoWay). Extended mode still maintains a current item, so CurrentCellChanged, Tapped, KeyDown, LoadingRow, and Sorting handlers keep working unchanged.

### The synchronization contract in code-behind

EditModeDataGrid\_SelectionChanged must translate grid selection into Model\_ReceivingLoad.IsSelected and **not** keep a competing selection list:

```
private void EditModeDataGrid_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (sender is not DataGrid grid || grid.ItemsSource is not
    IEnumerable<Model_ReceivingLoad> rows)
    {
        return;
    }

    // The grid is paginated, so only realized rows are represented here. Selection truth
    // lives on the row, and this pass mirrors the grid into it for the current page
    only.
    foreach (var row in rows)
    {
        row.IsSelected = grid.SelectedItems.Contains(row);
    }
}
```

Because this runs on every page change too, the mirrored state is always page-consistent while off-page selections survive on the model.

### Tri-state select-all header

```
<controls:DataGridTemplateColumn Width="50" CanUserResize="False" Tag="Checkbox">
    <controls:DataGridTemplateColumn.HeaderTemplate>
        <DataTemplate>
            <CheckBox
                MinWidth="0"
                Margin="0"
                HorizontalAlignment="Center"
```

```

        AutomationProperties.Name="Select all skids on this page"
        Click="SelectAllPageCheckBox_Click"
        IsChecked="{Binding IsPageSelectionState, Mode=OneWay}"
        IsThreeState="True" />
    </DataTemplate>
</controls:DataGridTemplateColumn.HeaderTemplate>
<controls:DataGridTemplateColumn.CellTemplate>
    <DataTemplate x:DataType="models:Model_ReceivingLoad">
        <CheckBox
            MinWidth="0"
            Margin="0"
            HorizontalAlignment="Center"
            VerticalAlignment="Center"
            AutomationProperties.Name="{x:Bind PartID, Mode=OneWay}"
            IsChecked="{x:Bind IsSelected, Mode=TwoWay}" />
    </DataTemplate>
</controls:DataGridTemplateColumn.CellTemplate>
</controls:DataGridTemplateColumn>

```

HeaderTemplate is not a DataTemplate over a row, so IsPageSelectionState must be a property on a bindable header view model reachable from the template, or the header state must be set imperatively in LoadingRow. Setting it imperatively from the SelectAllPageCheckBox\_Click handler is the lower-risk option and is recommended.

## Pending-change marker

Add a Grid inside the existing PartID cell template so a row carrying an uncommitted batch change is unmistakable, and keep the quality-hold foreground converter that is already applied there:

```

<DataTemplate x:DataType="models:Model_ReceivingLoad">
    <Grid Padding="12,0" ColumnSpacing="6">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>
        <Rectangle
            Grid.Column="0"
            Width="3"
            VerticalAlignment="Stretch"
            Fill="{ThemeResource AccentFillColorDefaultBrush}"
            Visibility="{x:Bind HasPendingBatchChange, Mode=OneWay}" />
        <TextBlock
            Grid.Column="1"
            VerticalAlignment="Center"
            Foreground="{Binding PartID, Converter={StaticResource
PartIDToQualityHoldTextColorConverter}}"
            Text="{x:Bind PartID, Mode=OneWay}" />
    </Grid>
</DataTemplate>

```

HasPendingBatchChange is new on Model\_ReceivingLoad. Because it is one of the few new members on a widely used model, keep it [JsonIgnore] like the other UI-only members there.

## Part 3 — Batch Edit Overlay Wireframe

Hosted as ContentDialog content in Module\_Receiving/Dialogs/Dialog\_Receiving\_BatchEdit.xaml, following the structural conventions of Dialog\_Receiving\_EditModeColumnChooser. The XML below is a schematic wireframe, not compilable XAML: it shows hierarchy, sizing, and control choices.

```
<!-- WIREFRAME: Dialog_Receiving_BatchEdit -->
<ContentDialog Title="Batch edit 3 skids"
    PrimaryButtonText="Apply to 3 skids"
    CloseButtonText="Cancel"
    DefaultButton="Primary"
    MinWidth="760" MaxWidth="980">

    <ScrollView>
        <StackPanel Spacing="14">

            <!-- Region A: context strip — which rows, which source -->
            <InfoBar IsOpen="True" Severity="Informational"
                Title="3 skids selected"
                Message="Source: Current Labels · MMC000659 · 236,634 lb" />

            <!-- Region B: reconciliation -->
            <Border Background="{ThemeResource LayerFillColorDefaultBrush}"
                BorderBrush="{ThemeResource CardStrokeColorDefaultBrush}"
                BorderThickness="1" CornerRadius="6" Padding="12">
                <Grid ColumnSpacing="16" RowSpacing="6">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="Auto"/> <!-- label -->
                        <ColumnDefinition Width="Auto"/> <!-- value -->
                        <ColumnDefinition Width="Auto"/> <!-- label -->
                        <ColumnDefinition Width="Auto"/> <!-- value -->
                        <ColumnDefinition Width="*" /> <!-- verdict -->
                    </Grid.ColumnDefinitions>
                    <TextBlock Grid.Column="0" Text="Selected total"/>
                    <TextBlock Grid.Column="1" FontWeight="SemiBold" Text="236,634 lb"/>
                    <TextBlock Grid.Column="2" Text="Expected total"/>
                    <NumberBox Grid.Column="3" Width="140" Value="{x:Bind ViewModel.ExpectedTotal,
Mode=TwoWay}"/>
                    <StackPanel Grid.Column="4" Orientation="Horizontal" Spacing="6">
                        <FontIcon Glyph="{x:Bind ViewModel.VarianceGlyph, Mode=OneWay}"/>
                        <TextBlock Text="{x:Bind ViewModel.VarianceText, Mode=OneWay}"/>
                    </StackPanel>
                </Grid>
            </Border>

            <!-- Region C: field list — tick to override -->
            <ItemsControl ItemsSource="{x:Bind ViewModel.Fields}">
                <ItemsControl.ItemTemplate>
                    <DataTemplate x:DataType="models:Model_BatchEditField">
                        <Grid Padding="0,4" ColumnSpacing="10" ColumnDefinitions="36,190,*,260">
                            <CheckBox Grid.Column="0" IsChecked="{x:Bind IsApplied, Mode=TwoWay}"/>
                            <TextBlock Grid.Column="1" VerticalAlignment="Center" Text="{x:Bind
Label}"/>
                            <TextBlock Grid.Column="2" VerticalAlignment="Center"
                                FontStyle="{x:Bind MixedFontStyle, Mode=OneWay}"
```

```

        Foreground="{x:Bind MixedForeground, Mode=OneWay}"
        Text="{x:Bind MixedCurrentValueText, Mode=OneWay}"/>

<!-- Editor swaps by field kind: text / lookup / numeric+operation -->
<Grid Grid.Column="3">
    <TextBox
        Visibility="{x:Bind IsTextKind, Mode=OneWay}"
        Text="{x:Bind NewTextValue, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"/>
    <sharedControls:Control_Shared_TypedLookupTextBox
        Visibility="{x:Bind IsLookupKind, Mode=OneWay}"
        LookupType="{x:Bind LookupType, Mode=OneWay}"
        WarehouseCode="002"
        InputValue="{x:Bind NewTextValue, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
        ValidationCompleted="FieldLookup_ValidationCompleted"/>
    <Grid Visibility="{x:Bind IsNumericKind, Mode=OneWay}"
        ColumnDefinitions="Auto,*" ColumnSpacing="6">
        <ComboBox Grid.Column="0" Width="110"
            ItemsSource="{x:Bind OperationOptions}"
            SelectedItem="{x:Bind SelectedOperation, Mode=TwoWay}"/>
        <NumberBox Grid.Column="1" Value="{x:Bind NewNumericValue,
Mode=TwoWay}"/>
    </Grid>
</Grid>
</Grid>
</DataTemplate>
</ItemsControl.ItemTemplate>
</ItemsControl>

<!-- Region D: validation pane – see Proposed_Validation_Safeguards.md -->
<InfoBar IsOpen="{x:Bind ViewModel.HasBlockers, Mode=OneWay}"
    Severity="Warning"
    Title="Fix before applying"
    Message="{x:Bind ViewModel.BlockerSummary, Mode=OneWay}"/>

<!-- Region E: preview -->
<Expander Header="Preview 7 changes" IsExpanded="True">
    <Grid RowDefinitions="Auto,*">
        <Grid Grid.Row="0" Padding="6,4" Background="{ThemeResource
LayerFillColorDefaultBrush}"
            ColumnDefinitions="120,140,180,180,*">
            <TextBlock Grid.Column="0" Text="Skid"/>
            <TextBlock Grid.Column="1" Text="Field"/>
            <TextBlock Grid.Column="2" Text="Before"/>
            <TextBlock Grid.Column="3" Text="After"/>
            <TextBlock Grid.Column="4" Text="Status"/>
        </Grid>
        <ListView Grid.Row="1" MaxHeight="220"
            ItemsSource="{x:Bind ViewModel.PreviewRows, Mode=OneWay}"
            SelectionMode="None"/>
    </Grid>
</Expander>

</StackPanel>

```

```
</ScrollView>
</ContentDialog>
```

## Width adaptivity

Below a 900px window width, Region C's four-column field grid must collapse to two rows per field (checkbox plus label on one line, current value and editor on the next). Add a `VisualStateGroup` to the dialog root with `AdaptiveTrigger MinWindowWidth="900"`, mirroring the adaptive pattern already in `View_Reprint_ModulePage.xaml`.

## Dialog result handling

`Dialog_Receiving_BatchEdit.xaml.cs` stays thin:

- Assign `XamlRoot = this.XamlRoot` before `ShowAsync`, which is mandatory for `ContentDialog` in WinUI 3.
- Return the field application set as the dialog result, or expose it from the view model and let the caller read it after `ShowAsync` returns `ContentDialogResult.Primary`.
- No business logic. No direct service calls. The existing `Dialog_Receiving_EditModeColumnChooser` is the pattern to copy.

## Part 4 — Reprint Batch Filter Chips

Add a second row beneath the existing eight-column filters Grid in `View_Reprint_ModulePage.xaml`. Change the outer row definitions only if the chip row must be inside the same Grid.

```
<StackPanel Grid.Row="0" Margin="0,8,0,0" Orientation="Horizontal" Spacing="6">
    <ToggleButton Content="{x:Bind ViewModel.CurrentBatchChipLabel, Mode=OneWay}"
        IsChecked="{x:Bind ViewModel.IsCurrentBatchFilterActive, Mode=TwoWay}"
        Visibility="{x:Bind ViewModel.HasBatchHandoff, Mode=OneWay}"
        ToolTipService.ToolTip="{x:Bind ViewModel.BatchHandoffTooltip,
Mode=OneWay}"/>

    <ToggleButton Content="Recently modified · 24h"
        IsChecked="{x:Bind ViewModel.IsRecent24hFilterActive, Mode=TwoWay}"/>

    <ToggleButton Content="Recently modified · 7d"
        IsChecked="{x:Bind ViewModel.IsRecent7dFilterActive, Mode=TwoWay}"/>

    <Button Content="Clear batch filter"
        Command="{x:Bind ViewModel.ClearBatchFilterCommand}"
        Visibility="{x:Bind ViewModel.HasBatchHandoff, Mode=OneWay}"/>
</StackPanel>
```

Batch context banner, inserted above the existing “Already queued” `InfoBar` by expanding the inner Grid from `RowDefinitions="Auto,Auto,*"` to `RowDefinitions="Auto,Auto,Auto,*"`:

```
<InfoBar Grid.Row="0"
    IsClosable="False"
    IsOpen="{x:Bind ViewModel.HasBatchHandoff, Mode=OneWay}"
    Message="{x:Bind ViewModel.BatchContextMessage, Mode=OneWay}"
    Severity="{x:Bind ViewModel.BatchContextSeverity, Mode=OneWay}"
    Title="{x:Bind ViewModel.BatchContextTitle, Mode=OneWay}"/>
```

`BatchContextSeverity` escalates to `Warning` when the batch did not balance, so an unbalanced label run is visually flagged at the top of the run.

## Two new columns

Header Grid becomes `ColumnDefinitions="36,*,*,140,90,*"` for Checkbox, Date, Part, Location, Qty, Reference. The `ListView.ItemTemplate` Grid must use the identical definition, because these are hand-aligned grids rather than an auto-generating `DataGrid`. Add Heat/Lot as a seventh column that is collapsed unless the chooser enables it.

## New Converters and Resources

Per the repository converter convention, converters live in `Module_Core/Converters`, are named `Converter_*`, handle null input defensively, and are registered in the consuming view's `UserControl.Resources`.

| Converter                                  | Purpose                                                                              | Registered in                        |
|--------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------|
| <code>Converter_BooleanToVisibility</code> | Already exists                                                                       | Reuse                                |
| <code>Converter_DecimalToString</code>     | Already exists, used for totals and variance                                         | Reuse                                |
| <code>Converter_MixedValueToDisplay</code> | Turns a <code>Model_BatchEditField</code> into value, (blank), or (mixed – N values) | Batch Edit dialog                    |
| <code>Converter_VarianceToBrush</code>     | Balanced / Over / Short to green, amber, red                                         | Edit Mode view, dialog, Reprint page |
| <code>Converter_VarianceToGlyph</code>     | Same states to a glyph                                                               | Edit Mode view, dialog               |

Variance state itself should be computed in the view model so the converters stay presentational and unit-testable.

## XAML Conventions and Risks

| Topic               | Rule                                                                                                                                                                                                         |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Existing columns    | Leave the existing {Binding} column templates alone. They are the shipped pattern inside DataGrid templates in this file, and rewriting them is churn outside this feature's scope.                          |
| New templates       | Always declare x:DataType and use x:Bind. TwoWay must be stated explicitly on x:Bind.                                                                                                                        |
| Code-behind         | Selection mirroring, dialog.XamlRoot, and checkbox click handlers only. No service calls, no validation logic, no persistence.                                                                               |
| Business logic      | Belongs in ViewModel_Receiving_EditMode and the new ViewModel_Receiving_BatchEdit.                                                                                                                           |
| Persistence         | Nothing new. Batch apply populates in-memory rows; SaveAsync persists through UpdateReceivingLoadsAsync or UpdateCurrentLabelDataAsync.                                                                      |
| User-facing strings | Add keys under Module_Receiving/Settings/ReceivingSettingsKeys.UiText and Module_Reprint/Settings/ReprintSettingsKeys rather than literals in XAML, matching the existing EditModeSaveAndFinishText pattern. |

## Risks

| Risk                                                                                              | Mitigation                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SelectionMode="Extended" changes existing single-row behavior that SelectedLoad consumers rely on | Verify every SelectedLoad consumer after the switch. Keep the checkbox as the authoritative bulk selector so any surprise in grid selection degrades to "no worse than today".    |
| Selection is lost on pagination                                                                   | Covered by the model-backed contract in BatchSkidEdit_UX_Specification.md. Add an explicit test that selects on page 1, pages to 2, and back, and asserts the count is unchanged. |
| Row definitions shifted by the Selection Bar insert                                               | One-time mechanical change; the empty-state overlay and footer both move by one row index. Verify the empty state still renders with the grid hidden.                             |
| ContentDialog cannot find a.XamlRoot                                                              | Set.XamlRoot explicitly before ShowAsync; failure here is the classic WinUI 3 dialog exception.                                                                                   |
| Concurrent edits during a modal batch change                                                      | The overlay is modal by design, so cell editing cannot race it.                                                                                                                   |